

Diseño de un Algoritmo de Una Sola Vía Basado en un Generador Pseudoaleatorio

Jaiber Mauricio Tabares Nieto – 20222020177
Cesar Augusto Ospina Gomez – 20222020059
Cristian Camilo Salamanca Escarraga – 20222020207
Laura Nathaly Páez Cifuentes – 20232020055
David Santiago Aldana Gonzalez – 20222020158

Repositorio: https://github.com/Salda1308/Ciencias_II_Repo

25 de junio de 2026

1. Introducción

El presente taller desarrolla un algoritmo de una sola vía apoyado en un generador pseudoaleatorio de números. El trabajo se organiza en dos etapas: primero se valida que el generador produzca valores entre 0 y 100 con probabilidades aproximadamente uniformes, y luego se utiliza ese generador como base para transformar mensajes de forma no reversible, tomando como referencia la idea general de funciones tipo *hash* como SHA.

El flujo general consiste en ingresar una semilla, alimentar con ella el generador pseudoaleatorio, obtener números de mezcla, permutar el mensaje, aplicar un proceso de arrastre con operaciones aritméticas y XOR, filtrar caracteres imprimibles y ordenar el resultado por código ASCII. La intención no es recuperar el mensaje original, sino producir una salida transformada que dependa de la semilla y del contenido ingresado.

2. Generador pseudoaleatorio utilizado

El generador implementado trabaja a partir de una semilla ingresada por el usuario. A partir de esta semilla se inicializan dos variables internas, p y q , que se actualizan en cada iteración mediante operaciones modulares. El uso de módulos grandes ayuda a producir cambios amplios en el estado interno del generador, mientras que la salida final se reduce al rango deseado mediante la operación módulo 101.

Listing 1: Código del generador pseudoaleatorio evaluado.

```

1 M1 = 1_000_000_007
2 M2 = 998_244_353
3 A = 6_271
4 B = 3_571
5
6 semilla = int(input("Ingresa una semilla: "))
7
8 p = semilla % M1 or 1
9 q = (semilla * 1_000_003 + 1) % M2
10
11 def generar():
12     global p, q
13
14     p = (A * p + q) % M1
15     q = (B * q + p * p) % M2
16
17     return (p + q) % 101
18
19 N = 100000000
20 conteo = [0] * 101
21
22 for _ in range(N):
23     conteo[generar()] += 1
24
25 print(f"\nResultados tras {N} iteraciones")
26 print(f"{'Numero':>8} {'Conteo':>8} {'Probabilidad':>14}")
27 print("-" * 34)
28
29 for num, cnt in enumerate(conteo):
30     prob = cnt / N
31     print(f"{num:>8} {cnt:>8} {prob:>14.4f}")
32
33 print("-" * 34)
34 print(f"{'Total':>8} {N:>8} {'1.0000':>14}")

```

3. Metodología de la prueba

Para evaluar el comportamiento del generador se utilizó la semilla 50 y se ejecutaron 100 000 000 iteraciones. En cada iteración se generó un número entre 0 y 100 y se aumentó el conteo correspondiente a ese valor.

Como existen 101 valores posibles, si la distribución fuera perfectamente uniforme, la probabilidad esperada para cada número sería:

$$P(x) = \frac{1}{101} \approx 0,00990099. \quad (1)$$

Por lo tanto, en una simulación de 100 000 000 iteraciones, el conteo esperado para cada valor es

aproximadamente:

$$\frac{100\,000\,000}{101} \approx 990\,099. \quad (2)$$

4. Resultados obtenidos

La ejecución del programa con semilla 50 mostró que todos los números del intervalo $[0, 100]$ aparecieron con una probabilidad cercana a 0,0099. Los conteos obtenidos estuvieron alrededor del valor esperado de 990 099 apariciones por número.

Algunos resultados representativos se muestran en la Tabla 1.

Numero	Conteo	Probabilidad
0	989752	0.0099
10	989207	0.0099
20	991230	0.0099
35	991570	0.0099
47	992314	0.0099
61	987775	0.0099
84	992369	0.0099
100	989815	0.0099

Cuadro 1: Muestra de resultados obtenidos tras 100 000 000 iteraciones.

Aunque los conteos no son exactamente iguales, las diferencias son pequeñas en comparación con el total de iteraciones. Esto es normal en un generador pseudoaleatorio, ya que la uniformidad esperada se cumple de manera aproximada y no como una repetición exacta del mismo conteo para cada valor.

5. Desarrollo del algoritmo de una sola vía

Después de verificar que el generador pseudoaleatorio produce números con probabilidades aproximadamente uniformes, se utiliza este componente como base para construir un algoritmo de una sola vía. En este procedimiento no se generan clave pública ni clave privada; la semilla se usa directamente para inicializar el generador y también para definir el estado inicial del proceso de transformación.

La idea del algoritmo es similar a una función de resumen: a partir de una entrada y una semilla se obtiene una salida transformada. Durante el proceso se mezclan permutaciones, multiplicaciones, operación XOR, módulo 256, filtrado de caracteres visibles y ordenamiento ASCII. Estas operaciones buscan destruir información de posición y contenido, por lo que el objetivo no es descifrar posteriormente, sino generar una salida de una sola vía.

Listing 2: Código del algoritmo de una sola vía basado en el generador pseudoaleatorio.

```
1 M1 = 1_000_000_007
2 M2 = 998_244_353
3 A = 6_271
4 B = 3_571
5
6 semilla = int(input("Ingresa la semilla: "))
7
8 p = semilla % M1 or 1
9 q = (semilla * 1_000_003 + 1) % M2
10
11 def generar():
12     global p, q
13     p = (A * p + q) % M1
14     q = (B * q + p * p) % M2
15     return (p + q) % 101
16
17 N = 10
18 numeros = []
19
20 for _ in range(N):
21     numeros.append(generar())
22
23 print(f"Arreglo de numeros generados: {numeros}")
24
25 palabra = input("Ingresa la palabra a transformar: ")
26
27 indices = sorted(range(len(palabra)), key=lambda i: numeros[i % N])
28 palabra_permutada = ''.join(palabra[i] for i in indices)
29
30 cifrado = ""
31 estado = semilla % 256 or 1
32 for i, char in enumerate(palabra_permutada):
33     clave = numeros[i % N]
34     estado = (ord(char) * clave + estado ^ clave) % 256
35     if 33 <= estado <= 126:
36         cifrado += chr(estado)
37
38 print(f"Texto cifrado: {cifrado}")
39
40 cifrado_ordenado = ''.join(sorted(cifrado, key=ord))
41 print(f"Texto cifrado ordenado (ASCII): {cifrado_ordenado}")
```

5.1. Flujo general del algoritmo

El funcionamiento del algoritmo puede resumirse en las siguientes etapas:

1. **Ingreso de la semilla.** La semilla se usa como valor inicial del generador pseudoaleatorio y como estado inicial del proceso de transformación.

2. **Generación del arreglo de claves.** Se generan 10 números pseudoaleatorios entre 0 y 100. Estos valores se reutilizan durante la permutación y la mezcla del mensaje.
3. **Permutación del mensaje.** Las posiciones de los caracteres de la palabra se reordenan según los números generados. Esto modifica la ubicación original de cada carácter.
4. **Mezcla con arrastre.** Cada carácter permutado se transforma usando multiplicación, suma, operación XOR y módulo 256. Además, el estado anterior influye en el siguiente carácter, por lo que un cambio en una parte del mensaje puede afectar resultados posteriores.
5. **Filtrado de caracteres no imprimibles.** Solo se conservan caracteres ASCII visibles entre 33 y 126. Los caracteres de control, espacios y símbolos no visibles se eliminan, haciendo que la salida pueda ser más corta que el mensaje original.
6. **Ordenamiento ASCII.** Finalmente, el resultado se ordena por valor ASCII para añadir una capa adicional de confusión posicional.

5.2. Resultados de ejecución

Con semilla 50 y mensaje `Hola Mundo esta es la clase de Ciencias`, el programa produjo la siguiente salida:

```
Arreglo de números generados: [42, 69, 96, 40, 26, 21, 92, 16, 29, 0]
Texto cifrado: B!wppo17ss
Texto cifrado ordenado (ASCII): !17Boppssw
```

Con la misma semilla 50, pero usando el mensaje `Hola Mundo esta es la clase de Ciencia`, se obtuvo:

```
Arreglo de números generados: [42, 69, 96, 40, 26, 21, 92, 16, 29, 0]
Texto cifrado: B>F66xeMAAc
Texto cifrado ordenado (ASCII): 66>AABFMcex
```

Al cambiar la semilla a 67 y usar nuevamente el mensaje `Hola Mundo esta es la clase de Ciencias`, los números generados y la salida cambiaron:

```
Arreglo de números generados: [26, 50, 77, 74, 2, 98, 23, 85, 39, 84]
Texto cifrado: =%*lGq}SDWWy]w@:h
Texto cifrado ordenado (ASCII): %*:=@DGSWW]hlqwy}
```

Estos resultados muestran que la semilla modifica la secuencia pseudoaleatoria y que pequeños cambios en el mensaje también producen salidas diferentes. Además, se observa que el texto resultante puede tener menor longitud que el texto original debido al filtrado de caracteres no imprimibles, lo cual refuerza la idea de una transformación de una sola vía y no de un cifrado pensado para recuperarse.

6. Conclusión general

En la primera parte se comprobó experimentalmente que el generador pseudoaleatorio distribuye los números del 0 al 100 con probabilidades cercanas entre sí, aproximadamente 0,0099 para cada valor. Esto permite usarlo como base para producir números de apoyo dentro del algoritmo de una sola vía.

En la segunda parte, el generador se integró dentro de un procedimiento que usa la semilla directamente para producir números pseudoaleatorios. Estos números controlan la permutación del mensaje, la mezcla con arrastre, el filtrado ASCII y el ordenamiento final. Al no existir una clave pública ni un proceso de descifrado, el algoritmo se entiende mejor como una transformación de una sola vía inspirada en funciones de resumen.

Con esto se completa el flujo de trabajo del taller: primero se valida el generador y luego se utiliza como componente central para construir el algoritmo propuesto. Aunque el procedimiento cumple el objetivo académico de combinar generación pseudoaleatoria, operaciones de mezcla y pérdida de información, para aplicaciones reales se deben usar funciones hash estandarizadas y revisadas públicamente, como las familias SHA.